

PRUEBA TÉCNICA OPA SAS

Desarrollador Semi Senior — Sector Financiero Solidario

Objetivo: evaluar conocimientos de backend, frontend, SQL, seguridad, integración de servicios, reglas de negocio, arquitectura y escalabilidad mediante la construcción de un módulo de créditos.

Nivel	Semi Senior
Tiempo recomendado	8–12 horas de trabajo efectivo
Base de datos	Preferiblemente Microsoft SQL Server
Backend recomendado	Node.js + TypeScript + Express
Frontend recomendado	JavaScript / TypeScript + React, Angular, Vue o Svelte

1. Contexto

Una entidad del sector financiero solidario desea implementar un pequeño módulo para administrar las solicitudes de crédito de sus asociados. El sistema debe permitir registrar, consultar, actualizar y cambiar el estado de los créditos mediante una API REST, utilizando SQL Server como motor de persistencia. Adicionalmente, cada vez que se cree un nuevo crédito, el sistema deberá notificar a un sistema externo mediante un webhook.

2. Alcance general

La solución debe incluir como mínimo:

- API REST.
- Base de datos SQL.
- CRUD de créditos.
- Frontend web.
- Integración mediante webhook.
- Seguridad de la API.
- Validaciones y reglas de negocio.
- Manejo consistente de errores.
- Documentación técnica.
- Propuesta de despliegue y escalabilidad.

3. Modelo de negocio

El producto principal de la prueba será un **crédito de asociado**. Cada crédito deberá contener como mínimo:

Campo	Descripción
id	Identificador único
numeroCredito	Número del crédito
identificacionAsociado	Identificación del asociado
nombreAsociado	Nombre del asociado
tipoCredito	Tipo de crédito

Campo	Descripción
valorSolicitado	Valor solicitado
tasaInteres	Tasa de interés
numeroCuotas	Número de cuotas
formaPago	Forma de pago
estado	Estado del crédito
fechaSolicitud	Fecha de solicitud
fechaActualizacion	Fecha de actualización

Estados sugeridos: SOLICITADO, EN_ESTUDIO, APROBADO, RECHAZADO, DESEMBOLSADO, CANCELADO. El candidato puede proponer un modelo diferente si lo justifica.

4. Base de datos

Debe crear el esquema SQL necesario. Como mínimo se espera una tabla **Creditos**. Se valorará positivamente una modelación con entidades como Asociados, Creditos e HistorialCredito.

Debe justificar llaves primarias, llaves foráneas, índices, tipos de datos, restricciones, campos obligatorios, manejo de fechas y estados.

Importante: los valores monetarios no deben manejarse como float/double. Utilice un tipo apropiado como DECIMAL(18,2).

5. API REST

Método	Endpoint	Descripción
POST	/api/creditos	Crear crédito
GET	/api/creditos	Consultar créditos; idealmente con paginació y filtros
GET	/api/creditos/{id}	Consultar un crédito
PUT/PATCH	/api/creditos/{id}	Actualizar información
PATCH	/api/creditos/{id}/estado	Cambiar estado
DELETE	/api/creditos/{id}	Eliminar o realizar borrado lógico

La consulta de créditos debería soportar, como mínimo, paginación y filtros por estado e identificación. Se valorará ordenamiento y búsqueda.

Ejemplo de creación:

```
POST /api/creditos
```

```
{ "identificacionAsociado": "1001234567", "nombreAsociado": "Juan Perez", "tipoCredito":
"LIBRE_INVERSION", "valorSolicitado": 15000000, "tasaInteres": 1.5, "numeroCuotas": 36, "formaPago":
"NOMINA" }
```

6. Reglas de negocio

- valorSolicitado > 0.
- tasaInteres >= 0.
- numeroCuotas > 0.
- Control de créditos duplicados según las condiciones definidas por el candidato.

- Control de transiciones de estado. Por ejemplo, un crédito no debería pasar directamente de RECHAZADO a DESEMBOLSADO.
- El candidato debe explicar dónde implementó las reglas de negocio y por qué.

7. Webhook — Integración externa

Cada vez que un crédito sea creado exitosamente, la aplicación deberá poder notificar a un sistema externo mediante un webhook.

```
POST https://sistema-externo.com/webhooks/creditos
```

```
{ "event": "credito.creado", "eventId": "8f7d...", "timestamp": "2026-09-07T18:00:00Z", "data": {  
  "id": 123, "numeroCredito": "CR-2026-000123", "identificacionAsociado": "1001234567",  
  "valorSolicitado": 15000000, "estado": "SOLICITADO" } }
```

Requisito clave: el webhook debe reutilizar la lógica existente de creación del crédito. No se debe duplicar la lógica de negocio ni realizar un segundo INSERT independiente. Debe quedar traza de la notificación por el WebHook.

El candidato debe explicar qué ocurre si el sistema externo no responde, devuelve error o está temporalmente fuera de servicio.

NOTAS:

Puede realizar la API para recibir las solicitudes de webhook o solo guardar la traza de que se notificó los créditos vía API.

8. Seguridad

La API debe estar asegurada y el candidato debe documentar y justificar los mecanismos utilizados.

9. Frontend

Construir una interfaz web para administrar créditos. El ingreso de usuarios no es obligatorio durante la prueba, aunque una implementación de autenticación será valorada positivamente.

- **Dashboard:** total de créditos y cantidades por estado.

- **Listado:** tabla con crédito, asociado, valor, tipo, estado y fecha; búsqueda, filtros y paginación.
- **Crear crédito:** formulario con validaciones.
- **Detalle:** información del crédito y su historial.
- **UX:** mensajes claros para éxitos, validaciones y errores de API.

10. Manejo de errores

La API debe utilizar códigos HTTP apropiados y respuestas consistentes. Por ejemplo: 200, 201, 400, 401, 403, 404, 409, 422 y 500 según corresponda.

```
{ "success": false, "error": { "code": "CREDITO_NOT_FOUND", "message": "El crédito no existe" } }
```

11. Auditoría y trazabilidad

Al tratarse de información financiera, se valorará que el candidato implemente o proponga un historial de cambios. Por ejemplo: crédito, estado anterior, estado nuevo, usuario, fecha y observación.

Debe explicar qué información debería poder modificarse y qué información debería considerarse inmutable en un sistema financiero.

12. Pruebas automatizadas

Se espera al menos un conjunto básico de pruebas para creación de crédito, validaciones, consulta inexistente, cambio de estado y transiciones inválidas. Se recomienda incluir una prueba que verifique la integración con el webhook.

13. Swagger / OpenAPI

Se recomienda documentar la API mediante Swagger/OpenAPI y permitir probar los endpoints principales desde la documentación.

14. Docker y ejecución

Es recomendable proporcionar un docker-compose.yml para facilitar la ejecución de frontend, backend y SQL Server. No es obligatorio, pero representa un diferenciador positivo.

15. Documentación requerida

- Código fuente.
- Script SQL.
- API REST.
- Frontend.
- Implementación del webhook.
- README.md con instrucciones de instalación y ejecución.
- Swagger/OpenAPI.
- Pruebas automatizadas.
- .env.example sin secretos reales.
- Diagrama de arquitectura.
- Documento corto con las principales decisiones técnicas.

16. Despliegue y arquitectura

El candidato debe explicar cómo expondría la aplicación en un ambiente productivo. Debe considerar, como mínimo, HTTPS, variables y secretos, backup de SQL Server, logs, health checks, monitoreo y CI/CD.

También debe presentar un diagrama sencillo de arquitectura y explicar qué componentes serían stateless y cuáles conservarían estado.

17. Escalabilidad

Responder: **Si el sistema inicialmente maneja 500 créditos mensuales, pero en dos años debe manejar 500.000 créditos mensuales y múltiples entidades cooperativas, ¿qué modificaría en su arquitectura?**

No es necesario implementar toda la arquitectura propuesta. Se evaluará la capacidad de identificar problemas futuros y justificar las decisiones.

18. Criterios de evaluación

Área	Peso
Backend / API REST	20%
SQL / modelamiento	5%
Reglas de negocio	5%
Webhook / integración	5%
Seguridad	5%
Frontend / UX	5%
Arquitectura / escalabilidad /Testing / documentación	5%
Socialización y justificación de la evaluación	50%
TOTAL	100%